Scaled-Weight Decomposition for L_2-Regularized Stochastic Gradient Descent on Sparse Data: Verification and Reproduction in C++17

Ngo Duy-Trang
Student ID 26125082

Final project – Introduction to Computer Science (CS161)

Abstract. L_2 regularization forces every step of stochastic gradient descent (SGD) to update the whole weight vector, even though a sparse sample has very few nonzero entries, so the cost of a step grows with the number of dimensions. Splitting the weights into a scale factor and a vector can make this cost depend only on the number of nonzeros. This paper verifies the technique for least-squares regression by implementing SGD in plain C++ in two forms (direct update and scaled update) and comparing it with scikit-learn's `SGDRegressor` to answer two questions: (RQ1) how does the per-step cost change with the number of dimensions, the density and the number of samples; (RQ2) can the C++ version reproduce scikit-learn on real data. On synthetic sparse data with one million dimensions (20 nonzeros per sample), the scaled form is more than 3000 times faster and its cost barely grows with the dimension; on fully dense data the two forms are nearly equal. On three house-price datasets, the weights of the C++ version match scikit-learn bit for bit and approach the Ridge solution. All 7 hypotheses set by us are supported, although the results were measured on a single machine and a single compiler. Code and data are released for reproduction at `https://github.com/sitingfake/sgd-weight-splitting-cpp`.

Keywords: SGD, L_2 regularization, sparse data, Ridge regression, scikit-learn, C++17.

1 Introduction

Many machine-learning problems today have very high dimensionality, from hundreds of thousands to millions, yet the data are sparse: each sample has only a few nonzero features. Typical examples are natural-language processing (Bag-of-Words, TF-IDF [1]), bioinformatics, graphs and recommender systems. At this scale, second-order methods and closed-form solutions based on matrix inversion are infeasible, because memory grows quadratically and time cubically with the dimension. Stochastic gradient descent (SGD) [2] therefore remains the main choice: it learns online, one sample at a time, with memory that grows only linearly with the dimension. To limit overfitting, L_2 regularization (Ridge) [3], which penalizes large weights, is added to the squared loss; the intercept is not penalized.

However, L_2 regularization creates a cost problem. Every update must shrink the whole weight vector, so the cost grows with the number of dimensions rather than with the number of nonzeros of the sample, even though the sample touches only a tiny part of the vector. The advantage of sparse data is thus lost exactly where it is needed most.

The remedy is known. Shalev-Shwartz et al. [4] (Pegasos) and Bottou [5] proposed splitting the weight vector into the product of a scalar and a vector. The shrinkage then becomes a single multiplication of the scalar, the error term touches only the nonzero positions of the sample, and the cost of a step depends on the number of nonzeros instead of the dimension. scikit-learn's `SGDRegressor` [6] is implemented in the same way. The technical details are given in Section 3.

Although the principle has been published, within the literature we surveyed two gaps remain. First, the foundational works experiment mainly with SVMs and the hinge loss, where many steps consist only of the shrinkage because the gradient is zero for samples outside the margin. With least-squares regression the residual is almost never zero, so every step also includes an update on the nonzero entries. This regime has not been compared systematically with the direct update across dimensions from one thousand to one million and densities from 0.1% to 100%. Second, scikit-learn has many internal details that affect numerical results, and we have not found a public comparison, with test code, showing that an independent C++17 implementation can match it step by step.

To fill these two gaps, we implement SGD in plain C++17 in two forms, direct update and scaled update, and compare it with scikit-learn's `SGDRegressor` to answer two research questions:

RQ1. How does the per-step cost of the two forms change with the number of dimensions, the density and the number of samples?

RQ2. Can the C++ version reproduce `SGDRegressor` on real data?

2 Data and Preprocessing

To answer RQ1 we use synthetic sparse data: in real data, the dimension n, the density and the number of samples m are usually tied together, whereas synthetic data let us vary one factor at a time through the three grids in Table 1. Each sample has exactly k distinct column indices chosen uniformly at random; the values are drawn from the standard normal distribution and then normalized to $\|x_i\|_2 = 1$ so that the gradient does not explode for large n. Labels are generated from a true weight vector drawn from the standard normal distribution, plus Gaussian noise with standard deviation 0.1. The generator uses the seed $[42, n, k, m]$, and

each data file has a SHA-256 hash.

Table 1. The three parameter grids of RQ1 ($\rho = k/n$ is the density).

Grid	Varied	Fixed
A	$n \in \{10^3, 10^4, 10^5, 10^6\}$	$k = 20$, $m = 10^4$
B	$k \in \{5, 50, 500, 2500, 5000\}$, i.e. ρ from 0.1% to 100%	$n = m = 5000$
C	$m \in \{10^3, 10^4, 10^5, 10^6\}$	$n = 10^4$, $k = 20$

To answer RQ2 we use three public house-price datasets, ranging from small ($m \approx 400$) to medium ($m \approx 20{,}000$) (Table 2). All three are fully dense, so RQ2 only checks equivalence and convergence, not speed-up. Taiwan [7] contains transactions in Xindian District with 6 features (transaction date, house age, distance to the MRT station, number of convenience stores, latitude, longitude); the column `No` is dropped. California [8] comes from the 1990 census with 8 features; the target is artificially capped, with about 4.7% of the observations at the ceiling. King County [9] contains transactions from May 2014 to May 2015 with 17 numeric features, dropping `id`, `date` and `zipcode` (a categorical identifier with very many levels); prices are strongly right-skewed (median 450,000 USD, maximum 7,700,000 USD).

Table 2. The real datasets of RQ2.

Dataset	m	n	Train/Test
Taiwan (UCI 477)	414	6	331/83
California Housing	20,640	8	16,512/4128
King County (OpenML 42092)	21,613	17	17,290/4323

The data are split 80/20 with `random_state=42`, `shuffle=True`; the row indices are saved so that C++, scikit-learn and Ridge share one partition. Z-score standardization is applied to both X and y, with the mean and standard deviation (ddof = 0) computed on the training set only, to avoid data leakage. For evaluation, predictions are transformed back to the original units before computing RMSE, MAE and R^2. The data are written in the binary format `SGDCSR01` (the first 32 bytes hold the identifier string, m, n and nnz; then `indptr` and `indices` as int32, `data` and y as float64), which avoids parsing CSV text and preserves every bit of float64 between Python and C++.

3 Method

The objective is

$$J(w, b) = \frac{1}{m} \sum_{i=1}^{m} \tfrac{1}{2} \big(w^\top x_i + b - y_i\big)^2 + \frac{\alpha}{2} \|w\|_2^2, \quad (1)$$

where the intercept b is not penalized. For standardized data, the closed-form Ridge solution [3] is $\hat{w}_{\text{ridge}} = (X^\top X + \alpha_{\text{ridge}} I)^{-1} X^\top y$. Since scikit-learn's `Ridge` minimizes $\|Xw - y\|_2^2 + \alpha_{\text{ridge}} \|w\|_2^2$ (not divided by m), multiplying (1) by $2m$ shows that the two objectives coincide when

$$\alpha_{\text{ridge}} = m_{\text{train}} \cdot \alpha_{\text{sgd}}. \quad (2)$$

Then $\hat{w}_{\text{ridge}}$ is the point that SGD approaches as the number of epochs $E \to \infty$.

At step t on sample (x_i, y_i), with learning rate $\eta_t = \eta_0 / t^{\texttt{power_t}}$ (the `invscaling` schedule), direct SGD (Algorithm 1) updates

$$w_{t+1} = (1 - \eta_t \alpha)\, w_t - \eta_t (\hat{y}_i - y_i)\, x_i. \quad (3)$$

Shrinking the whole w (the shrinkage factor is clipped from below at 0) is the $O(n)$ bottleneck, whereas the gradient is added at only $k \ll n$ nonzero positions.

Algorithm 1 Direct SGD

Input: CSR X, y, E, α, η_0, `power_t`, γ (intercept_decay)

$w \leftarrow 0,\ b \leftarrow 0,\ t \leftarrow 1$
for each epoch, each sample i in order **do**
  $p \leftarrow b + \sum_{j \in \text{supp}(x_i)} w_j x_{ij}$
  $g \leftarrow \text{clip}(p - y_i, \pm 10^{12})$
  $\eta \leftarrow \eta_0 / t^{\texttt{power_t}}$
  $w_j \leftarrow \max(0, 1 - \eta\alpha)\, w_j$ **for all** $j = 1..n$ ▷ $O(n)$
  $u \leftarrow -\eta g$
  if $u \neq 0$ **then**
    $w_j \leftarrow w_j + u\, x_{ij},\ j \in \text{supp}(x_i)$ ▷ $O(k)$
    $b \leftarrow b + \gamma u$
  end if
  $t \leftarrow t + 1$
end for
return w, b

Scaled SGD (Algorithm 2) sets $w = s \cdot v$ with $s_1 = 1$, $v_1 = 0$. The shrinkage becomes just $s \leftarrow \max(0, 1 - \eta_t \alpha)\, s$, costing $O(1)$, and the error term touches only k positions of v:

$$v_j \leftarrow v_j + \frac{u_t}{s} x_{ij}, \quad j \in \text{supp}(x_i), \qquad u_t = -\eta_t (\hat{y}_i - y_i), \quad (4)$$

where s is the value after shrinking. The prediction is computed indirectly as $\hat{y} = s\,(v^\top x_i) + b$. The cost of a step drops from $O(n)$ to $O(k)$, and the sequence of iterates is unchanged in exact arithmetic, because $s_{t+1} v_{t+1} = (1 - \eta_t \alpha)\, w_t + u_t x_i$. When s becomes too small, the division by s makes v blow up, so we renormalize $v \leftarrow s\, v,\ s \leftarrow 1$ when $s < 10^{-9}$. This step costs $O(n)$ but is rare: with $\alpha = 10^{-4}$ and $\eta_0 = 0.01$, about 4×10^9 steps are needed for s to reach the threshold. In the largest experiment (3×10^6 steps), $s \approx 0.91$ and the number of resets is 0, so this cost has no effect. The reset branch is tested separately with a forced configuration $\alpha = 10$.

The C++ version follows scikit-learn 1.9.1's Cython code in these respects: the (s, v) representation as `wscale`/`w`; the reset threshold 10^{-9}; the execution order *dot* → gradient clipping → shrinkage → add (the add happens only when $u \neq 0$); gradient clipping at $\pm 10^{12}$; b is not shrunk and uses `intercept_decay`; the `invscaling` schedule with t starting at 1 and continuing across epochs; sequential traversal when `shuffle=False`; no early stopping (`tol=None`).

For RQ1 we use the squared loss, $\alpha = 10^{-4}$, `fit_intercept=False`, `invscaling` with $\eta_0 = 0.01$, `power_t` $= 0.25$, $E = 3$, $w_0 = 0$. On Grid A we fit $\log T = \log c + \beta \log n$ to obtain the slope β (expected $\beta_{\text{direct}} \approx 1$, $\beta_{\text{scaled}} \approx 0$). For RQ2 we set `fit_intercept=True` (`intercept_decay=1`), keep α

Algorithm 2 Scaled SGD, $w = s \cdot v$

Input: as in Algorithm 1

$v \leftarrow 0,\ s \leftarrow 1,\ b \leftarrow 0,\ t \leftarrow 1$
for each epoch, each sample i in order **do**
  $p \leftarrow s \sum_{j \in \mathrm{supp}(x_i)} v_j x_{ij} + b$
  $g \leftarrow \mathrm{clip}(p - y_i, \pm 10^{12})$
  $\eta \leftarrow \eta_0 / t^{\texttt{power_t}}$
  $s \leftarrow \max(0, 1 - \eta\alpha)\, s$ ▷ $O(1)$
  if $s < 10^{-9}$ **then**
    $v \leftarrow s\, v,\ s \leftarrow 1$ ▷ $O(n)$, rare
  end if
  $u \leftarrow -\eta g$
  if $u \neq 0$ **then**
    $v_j \leftarrow v_j + (u/s)\, x_{ij},\ j \in \mathrm{supp}(x_i)$ ▷ $O(k)$
    $b \leftarrow b + \gamma u$
  end if
  $t \leftarrow t + 1$
end for
return $w = s \cdot v,\ b$ ▷ outside the timed loop

and the schedule, and retrain from scratch with $E \in \{1, 5, 20, 100, 500\}$. Besides RMSE, MAE and R^2 on the test set (original units), we measure three relative errors: Rel_W between w_{cpp} and w_{sk}, $d_{\mathrm{ridge}}(E)$ between $w(E)$ and $\hat{w}_{\mathrm{ridge}}$, both of the form $\|a-b\|_2/\|b\|_2$, and Rel_RMSE $= |\mathrm{RMSE}_{\mathrm{cpp}} - \mathrm{RMSE}_{\mathrm{sk}}|/\mathrm{RMSE}_{\mathrm{sk}}$. We set seven hypotheses, with quantitative criteria in Table 6: H1-a (speed-up for large n), H1-b (log-log slope), H1-c (dense data), H1-d (invariance to m), H1-e (Direct ≡ Scaled), H2-a (C++ ≡ scikit-learn), H2-b (same trajectory toward Ridge).

The experiments ran on an AMD Ryzen 9 8940HX, 16 GB DDR5 RAM, Windows 11; MinGW-W64 GCC 14.2.0 with `-std=c++17 -O2`; Python 3.12.9, NumPy 2.5.4, SciPy 1.18.1, scikit-learn 1.9.1. The code runs single-threaded by setting `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `MKL_NUM_THREADS` to 1. Only the algorithm loop is timed (`std::chrono::steady_clock`); file reading and the reconstruction of $w = s \cdot v$ are outside the timed region. Each measurement consists of 1 warm-up run and 7 repeats, and we report the median and the IQR. The final vector w is hashed with SHA-256 so that the compiler cannot eliminate the loop (dead-code elimination).

4 Results and Discussion

RQ1: per-step cost. Costs are in μs per update (median); the speed-up is $T_{\mathrm{direct}}/T_{\mathrm{scaled}}$. On all three grids, the weights of Direct and Scaled differ by a relative error of at most 7.44×10^{-15}, i.e. at the float64 rounding level (H1-e).

Dimension (Grid A, Table 3, Figure 1). The cost of Direct grows with n with slope $\beta_{\mathrm{direct}} = 1.009$ ($R^2 > 0.9999$), exactly $O(n)$. The cost of Scaled is almost flat (≈ 0.036–$0.037\,\mu$s from $n = 10^3$ to 10^5), with $\beta_{\mathrm{scaled}} = 0.123$. At $n = 10^6$ the speed-up reaches 3446×: training 3 epochs on 10^4 samples drops from 6.72 s to 1.95 ms. The cost of Scaled rises slightly to $0.065\,\mu$s at $n = 10^6$; a plausible explanation is that the array v (8 MB) exceeds the L2 cache (not confirmed with hardware counters).

Table 3. Grid A ($k = 20$, $m = 10^4$): μs per update.

n	Direct	Scaled	sklearn	Speed-up
10^3	0.2578	0.0363	0.0615	7.10×
10^4	2.1514	0.0369	0.0646	58.27×
10^5	20.9877	0.0372	0.0818	564.24×
10^6	223.8895	0.0650	0.1215	**3446.04**×

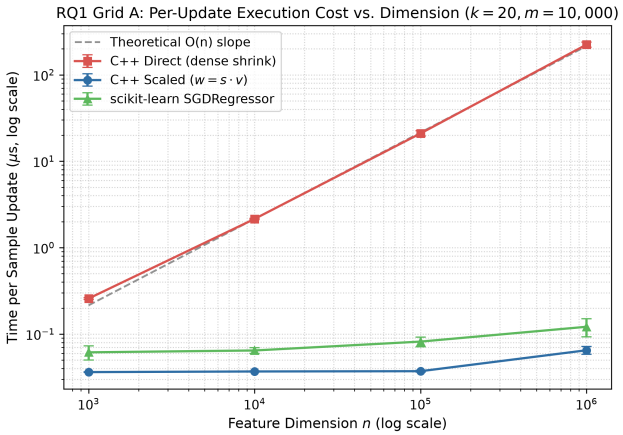


Figure 1. Grid A: per-update cost versus dimension n (log-log, $k = 20$, $m = 10^4$). Dashed line: theoretical $O(n)$ slope.

Density (Grid B, Table 4, Figure 2). The advantage shrinks as the density ρ grows: 54.99× at 0.1%, 18.95× at 1%, 2.59× at 10%, and 1.18× at 100%. At 100% density the two forms have the same order $O(n)$; Scaled is not slower than Direct, but the 1.18× difference is small compared with the spread of the measurements (Figure 2), so we conclude only that no significant degradation is observed.

Table 4. Grid B ($n = m = 5000$): μs per update.

k	ρ	Direct	Scaled	sklearn	Speed-up
5	0.1%	1.1053	0.0201	0.0699	54.99×
50	1%	1.1253	0.0594	0.1393	18.95×
500	10%	1.7420	0.6716	0.8989	2.59×
2500	50%	4.1113	3.1202	5.0361	1.32×
5000	100%	7.2009	6.0977	7.1697	1.18×

Sample count (Grid C, Figure 3). As m grows from 10^3 to 10^6, the cost of Scaled varies only within 0.0352–$0.0380\,\mu$s (max/min ratio 1.08) and the speed-up stays at 57×–60×. The per-step cost does not depend on the size of the sample set.

Comparison with scikit-learn. C++ Scaled is faster than `SGDRegressor` by about 1.7–2.2× on Grid A and 1.2–3.5× on Grid B. Note that `SGDRegressor` was timed through the Python interface, so the measurement conditions are not fully identical to those of C++; we regard this as a reference comparison.

RQ2: equivalence and convergence. On all 15 pairs (3 datasets × 5 values of E), every deviation (Rel_W, Rel_RMSE, and the MAE and R^2 deviations) is exactly 0, i.e. bit-for-bit agreement in this measurement environment (H2-a). At every epoch, $|d_{\mathrm{cpp}} - d_{\mathrm{sk}}|/d_{\mathrm{sk}} = 0$, so the two models follow the same



Figure 2. Grid B: per-update cost versus density k/n ($n = m = 5000$). Error bars show the spread across repeated runs.

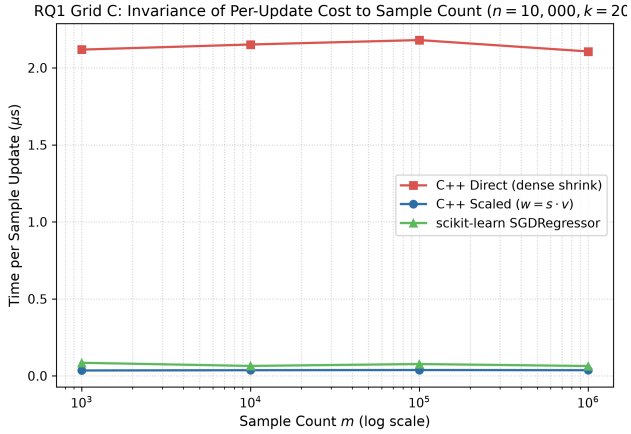


Figure 3. Grid C: per-update cost versus number of samples m ($n = 10^4$, $k = 20$).

trajectory toward the Ridge solution (H2-b). The distance d_{ridge} decreases monotonically with E on all three datasets (Table 5, Figure 4), but with `invscaling`, SGD has not reached the Ridge solution after 500 epochs: 0.71% remains for Taiwan, 1.55% for California and 4.99% for King County. The test RMSE at $E = 500$ is very close to Ridge on Taiwan (7.3187 vs. 7.3146) and California (0.7444 vs. 0.7455); on King County it differs by about 0.5% (215,620 vs. 214,473 USD). The RMSE, MAE and R^2 curves versus epochs are in the Appendix. In running time, at $E = 500$ C++ Scaled takes 3.75 ms versus 8.06 ms (Taiwan), 225.9 versus 395.8 ms (California) and 305.8 versus 461.2 ms (King County), i.e. about 1.5–2.2× faster than `SGDRegressor`. For small E the ratio is much larger (Taiwan, $E = 1$: 0.93 ms versus 0.01 ms), because the total time of `SGDRegressor` includes a fixed cost independent of E; the figure at $E = 500$ therefore reflects the true training cost better.

Hypothesis outcomes. All 7 of 7 hypotheses are supported under the measurement conditions of this report (Table 6).

Discussion. Direct must read and write the whole of v (8 MB when $n = 10^6$) for every sample, whereas Scaled touches only $k = 20$ entries. The cost therefore no longer scales with n, consistent with the slope of

Table 5. Relative distance to the Ridge solution $d_{\text{ridge}}(E)$, and quality at $E = 500$ (C++ Scaled and scikit-learn give identical values). RMSE in original units: Taiwan 10,000 NTD/Ping, California 100,000 USD, King County USD.

	Taiwan	California	King County
$d_{\text{ridge}}(E{=}1)$	0.4735	1.2310	0.0869
$d_{\text{ridge}}(E{=}5)$	0.2796	0.3775	0.0714
$d_{\text{ridge}}(E{=}20)$	0.1008	0.0273	0.0632
$d_{\text{ridge}}(E{=}100)$	0.0108	0.0217	0.0565
$d_{\text{ridge}}(E{=}500)$	0.0071	0.0155	0.0499
RMSE, Ridge	7.3146	0.7455	214,473.46
RMSE, SGD ($E{=}500$)	7.3187	0.7444	215,619.85
R^2, Ridge	0.6811	0.5758	0.6957
R^2, SGD ($E{=}500$)	0.6807	0.5771	0.6925

Table 6. Verdict on the 7 hypotheses set by us.

ID	Acceptance criterion	Observed
H1-a	$T_{\text{dir}}/T_{\text{sc}} \geq 2$ at $n \geq 10^4$, monotonically increasing	58.27×; 564.24×; 3446.04×
H1-b	$\beta_{\text{dir}} \in [0.7, 1.1]$, $\beta_{\text{sc}} \leq 0.35$	1.009; 0.123
H1-c	$0.5 \leq T_{\text{dir}}/T_{\text{sc}} \leq 3$ at 100%	1.18×
H1-d	max / min of $T_{\text{sc}} \leq 1.30$ (Grid C)	1.08
H1-e	$\|w_{\text{sc}} - w_{\text{dir}}\|/\|w_{\text{dir}}\| < 10^{-10}$	$\leq 7.44 \times 10^{-15}$
H2-a	Rel_W $< 10^{-10}$, Rel_RMSE $< 10^{-6}$	0.00 (15/15)
H2-b	$\|d_{\text{cpp}} - d_{\text{sk}}\|/d_{\text{sk}} < 10^{-6}$	0.00

0.12, and most of the gain probably comes from reducing the amount of memory accessed rather than only from reducing the number of arithmetic operations. The benefit is clear when $\rho \lesssim 1\%$ ($\geq 18\times$) and drops sharply around $\rho \approx 10\%$ (2.59×). Near 100% density, access through column indices may remove the SIMD vectorization available to dense arrays (a conjecture, not measured), so for fully dense data other approaches, such as mini-batch updates with BLAS, should be used. In practice, a cost of about 0.065 μs per sample corresponds to about 15 million samples per second on a single CPU thread at $n = 10^6$, $k = 20$, the same order as the throughput required by high-rate sparse data streams (such as CTR prediction or spam filtering). The C++ version does not need Python; its potential for embedded devices remains to be verified and was not measured here.

5 Conclusion and Limitations

This paper verifies the $w = s \cdot v$ technique for L_2-regularized least-squares regression with an independent C++17 implementation. For RQ1, at $n = 10^6$, $k = 20$, the scaled form is 3446× faster (0.065 versus 223.89 μs per step); the log-log slope is 0.123 (Scaled) versus 1.009 (Direct), matching $O(k)$ and $O(n)$. The cost does not change as the number of samples grows (1.08×), while the advantage narrows with density and is only 1.18× at 100%, with no slowdown observed. For RQ2, the C++ version matches `SGDRegressor` bit for bit on 15/15 pairs (3 datasets × 5 epoch budgets) and approaches the Ridge solution in the same way (d_{ridge}

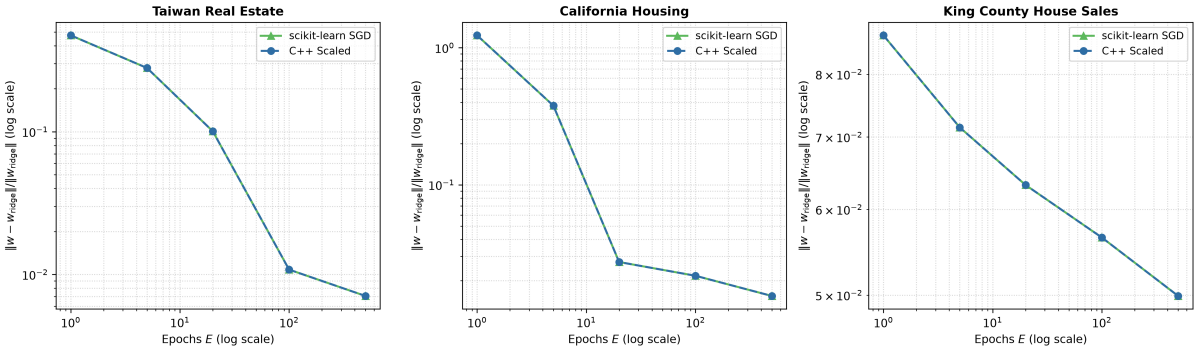


Figure 4. Relative distance to the Ridge solution versus number of epochs E (log-log). The curves of scikit-learn and C++ Scaled coincide.

is 0.7%–5% after 500 epochs).

These conclusions should be read within the scope of the experiments:

Measurement environment. All measurements were taken on one machine (Ryzen 9 8940HX), one compiler (GCC 14.2.0, `-O2`) and one operating system (Windows 11, which has no standard `taskset`, so the process could not be pinned to a single core; the spread remains considerable at high density). Absolute times will differ on other machines, and the slight rise at $n = 10^6$ depends on the cache size. The relative speed-up and the log-log slopes are properties of the algorithm and are likely to hold, but have not been verified on other architectures. The "bit-for-bit" claim also applies only to this environment (same compiler, optimization flags and library versions).

Data. The RQ1 data have uniformly random sparsity, whereas real data (TF-IDF, Bag-of-Words) usually follow a Zipf distribution [10], which can change the cache access pattern even though the per-step complexity $O(k)$ is unchanged. The RQ2 data are small and dense ($n \in \{6, 8, 17\}$), so they only test equivalence and convergence, not the speed-up on real sparse data.

Algorithmic scope. Only sequential single-threaded SGD is considered (Hogwild! [11] is not covered), without early stopping, and with the `invscaling` schedule on standardized data. The `optimal` schedule and the forced-reset configuration were used only in unit tests.

Future work is to measure on real sparse data with a Zipf distribution (for example MovieLens or text classification), to try more architectures and compilers, and to extend the study to other loss functions.

AI Usage Declaration

The author used two generative AI tools and no others. **Gemini 3.8 Flash** (Agent Mode in the Antigravity IDE) assisted with programming: the C++17 code skeleton (`sgd_direct.hpp`, `sgd_scaled.hpp`, `csr.hpp`, `timer.hpp`), the Python scripts (`gen_rq1.py`, `prep_rq2.py`, `binio.py`, `bench_sklearn.py`), the 38 `pytest` tests, the `Makefile` and table export. **Claude Sonnet 5.5** (Anthropic, free web plan) assisted with writing: section layout, editing and condensing the prose, and standardizing terminology. The author takes full responsibility for the content, mathematical correctness, data and conclusions; AI is not a co-author. All code, formulas and procedures proposed by AI were checked by the author: auditing scikit-learn's Cython code (`docs/sklearn_source_audit.md`), compiling and running everything locally (binary data checked by SHA-256), and verifying the mathematical derivations.

References

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] H. Robbins and S. Monro, "A stochastic approximation method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.

[3] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[4] S. Shalev-Shwartz, Y. Singer, N. Srebro and A. Cotter, "Pegasos: Primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[5] L. Bottou, "Stochastic gradient descent tricks," in *Neural Networks: Tricks of the Trade*, 2nd ed., Springer LNCS 7700, pp. 421–436, 2012.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] I.-C. Yeh and T.-K. Hsu, "Building real estate valuation models with comparative approach through case-based reasoning," *Applied Soft Computing*, vol. 65, pp. 260–271, 2018.

[8] R. K. Pace and R. Barry, "Sparse spatial autoregressions," *Statistics & Probability Letters*, vol. 33, no. 3, pp. 291–297, 1997.

[9] J. Vanschoren, J. N. van Rijn, B. Bischl and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.

[10] G. K. Zipf, *Human Behavior and the Principle of Least Effort*. Cambridge, MA: Addison-Wesley, 1949.

[11] F. Niu, B. Recht, C. Ré and S. J. Wright, "Hogwild!: A lock-free approach to parallelizing stochastic gradient descent," in *Advances in Neural Information Processing Systems 24*, pp. 693–701, 2011.

Appendix: Prediction Quality versus Epochs

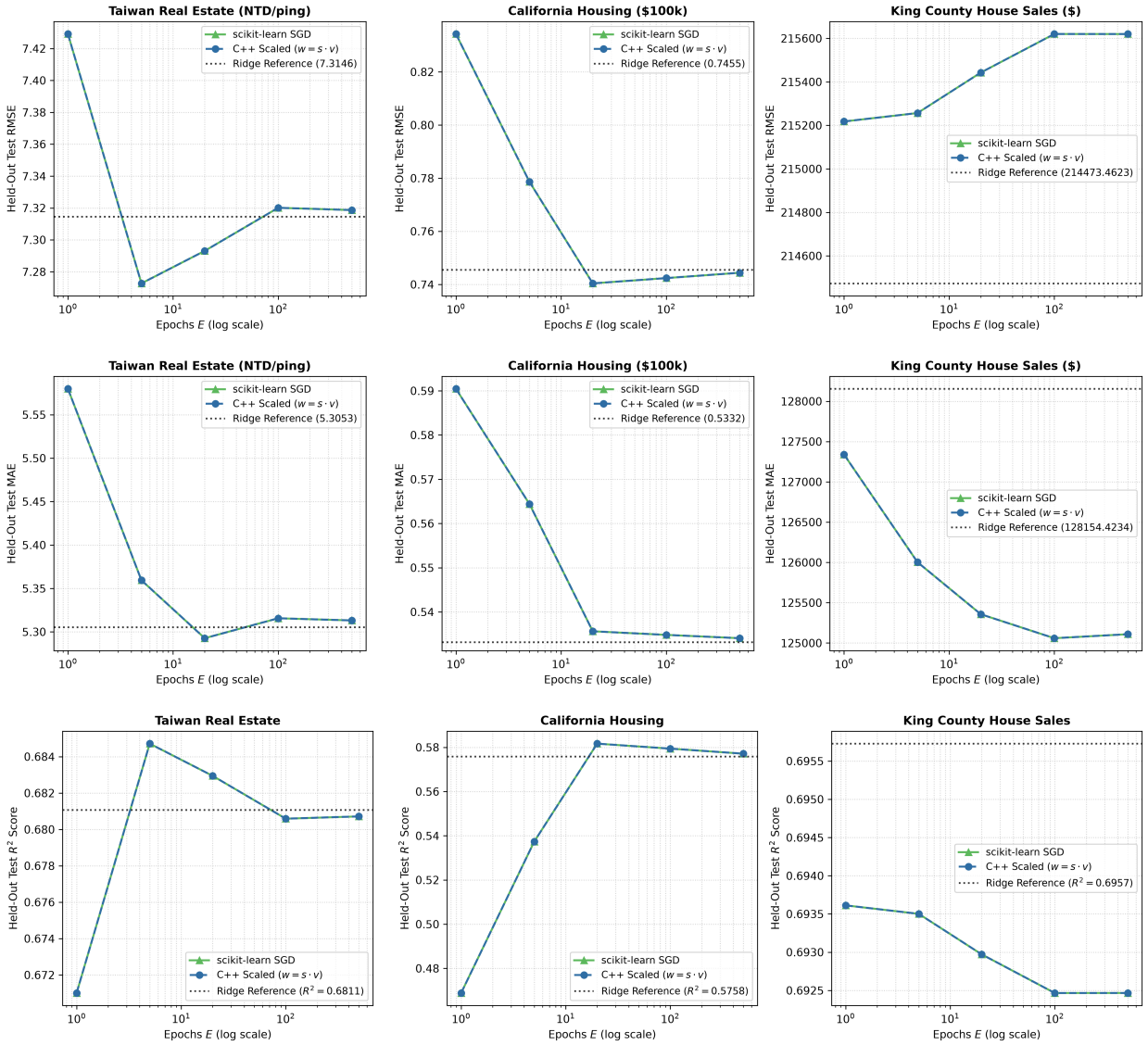


Figure 5. Test RMSE, MAE and R^2 versus number of epochs E (original units). The curves of scikit-learn and C++ Scaled coincide; the dotted line is the Ridge solution.